Pontifícia Universidade Católica de Campinas
Escola Politécnica - Faculdade Engenharia de Software

Avaliação Prática 1

Jogo da Velha com Agente Inteligente Minimax

Grupo 4:
Anderson lucas do nascimento gondim RA: 24787293
Filipe Ribeiro Simões RA: 24007657

Campinas, 25 de março de 2026

Sumário

Sumário 1
1. Introdução 2
2. Desenvolvimento 3
  2.1 Modelo de Sistema 4
    2.1.1 O jogo e suas regras 4
    2.1.2 Representação do tabuleiro 4
  2.2 Algoritmo Minimax 4
    2.2.1 Histórico 4
    2.2.2 Como o algoritmo funciona 5
    2.2.3 Game Tree e a relação com Busca 6
    2.2.4 Tipo de agente segundo Russell e Norvig 6
  2.3 Resultados Obtidos 7
3. Conclusão 8
4. Referências 9

Campinas, 25 de março de 2026

1. Introdução

Este relatório apresenta o desenvolvimento de um agente inteligente para o Jogo da Velha, usando o algoritmo Minimax. O trabalho foi desenvolvido a partir do código base da prática anterior, onde já tínhamos o jogo funcionando entre dois jogadores humanos. Aqui, a ideia foi substituir um dos jogadores por um agente que consegue tomar decisões sozinho.

O Jogo da Velha foi escolhido como ambiente de teste justamente porque é simples o suficiente para explorar toda a árvore de jogadas possíveis sem precisar de muita capacidade computacional, mas ao mesmo tempo tem complexidade suficiente para ilustrar os conceitos de Busca Competitiva, Game Tree e o próprio Minimax.

Durante o desenvolvimento, a parte que mais deu trabalho foi entender a lógica de alternância entre os nós MAX e MIN dentro da recursão, especialmente garantir que o backtracking estava desfazendo as jogadas simuladas corretamente antes de testar a próxima posição.

2. Desenvolvimento

2.1 Modelo de Sistema

2.1.1 O jogo e suas regras

O Jogo da Velha funciona num tabuleiro de 3x3 posições. Dois jogadores se alternam marcando uma posição vazia com seu símbolo — no nosso caso, "X" para o usuário e "O" para o agente. Quem conseguir alinhar três símbolos iguais em linha, coluna ou diagonal primeiro vence.

As regras que implementamos foram:

- Cada turno, o jogador da vez escolhe uma posição vazia do tabuleiro.
- Não é permitido marcar uma posição já ocupada.
- Vence quem completar uma linha, coluna ou diagonal primeiro.
- Se o tabuleiro encher sem nenhum vencedor, o jogo termina em empate.
- O usuário (X) sempre começa.

2.1.2 Representação do tabuleiro

Mantivemos a estrutura da prática anterior: o tabuleiro é uma lista de listas 3x3 em Python. Cada célula pode ter " " (vazio), "X" ou "O". Essa representação é simples e funciona bem para o Minimax porque dá para modificar e desfazer posições diretamente na mesma estrutura, sem precisar criar cópias a cada simulação.

2.2 Algoritmo Minimax

2.2.1 Histórico

O Minimax tem origem num artigo de Claude Shannon de 1950, "Programming a Computer for Playing Chess", onde ele propôs usar uma função de avaliação para medir o valor de cada posição do jogo e uma árvore de possibilidades para escolher o melhor movimento. A ideia central é simples: o agente assume que o adversário também vai jogar

da melhor forma possível, então ele precisa maximizar o próprio resultado levando isso em conta.

Um marco histórico do Minimax foi o Deep Blue, computador da IBM que venceu o campeão mundial de xadrez Garry Kasparov em 1997. O Deep Blue usava uma versão otimizada do Minimax com Poda Alpha-Beta, capaz de analisar uma quantidade enorme de jogadas por minuto. Isso mostrou que esse tipo de algoritmo podia competir de igual para igual com humanos mesmo em jogos muito complexos.

2.2.2 Como o algoritmo funciona

O Minimax funciona simulando todas as jogadas possíveis a partir do estado atual do tabuleiro, formando uma árvore de decisões chamada Game Tree. Em cada nível dessa árvore, o algoritmo alterna entre dois papéis:

- Nó MAX: é a vez do agente, que quer o maior valor possível.
- Nó MIN: é a vez do usuário, e o algoritmo assume que ele vai jogar da melhor forma — ou seja, tentando minimizar o valor do agente.

Quando chega num estado terminal (vitória, derrota ou empate), uma função de utilidade atribui um valor numérico àquele estado. No nosso projeto usamos:

- +10 menos a profundidade: agente ganhou. O desconto pela profundidade faz o agente preferir vitórias mais rápidas.
- -10 mais a profundidade: usuário ganhou. O acréscimo faz o agente "preferir" perder mais tarde do que cedo.
- 0: empate.

A estrutura central do código ficou assim:

```python
# algoritmo minimax - vai testar todas as jogadas possiveis e retornar o valor do melhor caminho
# maximizador = agente (O), minimizador = usuario (X)
def minimax(tabuleiro, profundidade, maximizando):

    # casos de parada
    if verifica_vitoria(tabuleiro, "O"):
        return 10 - profundidade  # ganhamos, buscar ganhar rapido

    if verifica_vitoria(tabuleiro, "X"):
        return -10 + profundidade  # perdemos, buscar demorar mais pra perder

    if verifica_empate(tabuleiro):
        return 0

    if maximizando:
        # vez do agente - quer o maior valor possivel
        melhor = float('-inf')
        for (l, c) in posicoes_vazias(tabuleiro):
            tabuleiro[l][c] = "O"
            resultado = minimax(tabuleiro, profundidade + 1, False)
            tabuleiro[l][c] = " "  # desfaz pra testar a proxima opcao
            if resultado > melhor:
                melhor = resultado
        return melhor

    else:
        # vez do usuario - assume que ele vai jogar o melhor possivel
        melhor = float('inf')
        for (l, c) in posicoes_vazias(tabuleiro):
            tabuleiro[l][c] = "X"
            resultado = minimax(tabuleiro, profundidade + 1, True)
            tabuleiro[l][c] = " "
            if resultado < melhor:
                melhor = resultado
        return melhor
```

```python
# percorre as jogadas possiveis e escolhe a de maior valor minimax
def jogada_agente(tabuleiro):
    melhor_val = float('-inf')
    jogada = None

    for (l, c) in posicoes_vazias(tabuleiro):
        tabuleiro[l][c] = "O"
        val = minimax(tabuleiro, 0, False)
        tabuleiro[l][c] = " "

        if val > melhor_val:
            melhor_val = val
            jogada = (l, c)

    return jogada
```

Campinas, 25 de março de 2026

Um detalhe importante é o backtracking: depois de simular cada jogada, o código desfaz a alteração no tabuleiro antes de testar a próxima posição. Isso evita precisar criar cópias do tabuleiro a cada chamada recursiva.

2.2.3 Game Tree e a relação com Busca

Nos algoritmos de busca que estudamos ao longo da disciplina, o objetivo é encontrar um caminho entre um estado inicial e um estado objetivo. No caso dos jogos, a lógica é parecida, mas tem uma diferença importante: não estamos procurando qualquer caminho até a vitória, mas sim o próximo movimento que, considerando que o adversário também joga bem, nos dá a melhor chance de vencer.

A Game Tree é justamente essa representação: cada nó é um estado do tabuleiro, cada aresta é uma jogada possível, e as folhas são os estados terminais. No Jogo da Velha, o número de folhas é limitado (no máximo 9 jogadas por partida), então dá para explorar toda a árvore sem precisar de heurísticas de corte.

Como descrito por Barth (2018): *"Não estamos procurando por um caminho. Apenas pelo próximo movimento (espera-se que este movimento me leve à vitória). Meus melhores movimentos dependem dos movimentos do meu adversário."* Essa frase resume bem a diferença entre busca convencional e busca competitiva.

Os elementos da busca adaptados para o contexto de jogos são: estado inicial (tabuleiro vazio e jogador que começa), operadores (marcar uma posição vazia), estados objetivo (tabuleiro com três símbolos alinhados) e função de utilidade (o valor numérico que avalia cada estado terminal).

2.2.4 Tipo de agente segundo Russell e Norvig

Russell e Norvig descrevem quatro tipos de programas de agentes: reflexo simples, reflexo baseado em modelo, baseado em objetivo e baseado em utilidade. O agente que desenvolvemos se enquadra como agente baseado em utilidade.

A justificativa é que ele não só conhece seu objetivo (vencer), mas avalia numericamente cada estado possível através da função de utilidade (+10, -10 ou 0). Com base nesses valores, ele escolhe a ação que maximiza o resultado esperado. Além disso, ele mantém um modelo interno do estado do jogo e simula as jogadas futuras antes de decidir — o que descarta o enquadramento como agente de reflexo simples.

Um agente baseado apenas em objetivo buscaria qualquer caminho para a vitória sem avaliar a qualidade da jogada. Já o nosso agente distingue entre "ganhar rápido" e "ganhar devagar", e entre "perder cedo" e "perder tarde" — essa preferência entre estados é justamente o que caracteriza um agente baseado em utilidade.

2.3 Resultados Obtidos

Testamos o agente jogando contra ele várias vezes, tentando diferentes estratégias. Abaixo está um exemplo de partida onde o agente venceu:

```
Jogo da Velha - Voce (X) vs Agente (O)
Linhas e colunas de 1 a 3

-------------
|   |   |   |
-------------
|   |   |   |
-------------
|   |   |   |
-------------
```

```
Sua vez:
Linha (1-3): 1
Coluna (1-3): 3
-------------
|   |   | X |
-------------
|   |   |   |
-------------
|   |   |   |
-------------
```

```
Agente pensando...
Agente jogou em (2, 2)
-------------
|   |   | X |
-------------
|   | O |   |
-------------
|   |   |   |
-------------

Sua vez:
Linha (1-3): 1
Coluna (1-3): 2
-------------
|   | X | X |
-------------
|   | O |   |
-------------
|   |   |   |
-------------
```

```
Agente pensando...
Agente jogou em (1, 1)
-------------
| O | X | X |
-------------
|   | O |   |
-------------
|   |   |   |
-------------

Sua vez:
Linha (1-3): 3
Coluna (1-3): 1
-------------
| O | X | X |
-------------
|   | O |   |
-------------
| X |   |   |
-------------
```

```
Agente pensando...
Agente jogou em (3, 3)
-------------
| O | X | X |
-------------
|   | O |   |
-------------
| X |   | O |
-------------
O agente ganhou!
```

Campinas, 25 de março de 2026

No exemplo acima, o usuário tentou montar uma diagonal mas o agente identificou que precisava garantir a coluna 1 para si e fez isso antes de o usuário conseguir bloquear.

Nos testes, percebemos que quando jogamos tentando montar duas ameaças ao mesmo tempo (o chamado "fork"), o agente consegue identificar e bloquear pelo menos uma delas, ou montar o próprio fork antes. Nos casos em que os dois lados jogaram da melhor forma possível, o resultado sempre foi empate — o que é o comportamento esperado para o Jogo da Velha com Minimax.

Campinas, 25 de março de 2026

3. Conclusão

O trabalho mostrou na prática como o algoritmo Minimax funciona. A maior dificuldade foi entender a recursão com backtracking: a ideia de simular a jogada, chamar o minimax recursivamente e depois desfazer a jogada para testar a próxima opção parece estranha no início, mas depois de alguns testes ficou mais claro.

O resultado foi um agente que nunca perde. Quando jogamos bem, empata; quando erramos, ele aproveita. Isso confirma o que a teoria diz: o Minimax garante o resultado ótimo dado que o adversário também jogue de forma ótima.

A relação com os conceitos de busca ficou mais clara durante a implementação. A Game Tree é literalmente a árvore de busca do jogo, e o Minimax é o algoritmo que percorre essa árvore escolhendo o melhor caminho — não qualquer caminho, mas o melhor considerando que o adversário também vai tentar nos prejudicar.

Campinas, 25 de março de 2026

4. Referências

- BARTH, Fabrício Jailson. Jogos de Tabuleiro e Busca Competitiva. Disciplina de Sistemas Inteligentes. Março de 2018. Disponível em: https://fbarth.net.br/materiais/docs/ia/buscaCompetitiva.pdf. Acesso em: 25 mar. 2026.

- RUSSELL, Stuart; NORVIG, Peter. Inteligência Artificial: uma abordagem moderna. 3. ed. Rio de Janeiro: Elsevier, 2010.

- SHANNON, Claude. Programming a Computer for Playing Chess. Philosophical Magazine. Londres, v. 41, n. 314, p. 256-275, 1950.

- SOUZA, Diego Marcelino. Jogo da Velha com Inteligência Artificial. 2014. Trabalho acadêmico — Universidade Federal do Espírito Santo, Vitória, 2014.

Campinas, 25 de março de 2026